

RELATÓRIO TÉCNICO: SISTEMA DE ORGANIZAÇÃO DE ATIVIDADES

Integrantes: Pedro Henrique, Samir Marinho e Diego Araujo

Data: 10 de junho de 2026

1. Descrição Geral do Sistema

O sistema desenvolvido consiste em uma ferramenta de gerenciamento de tarefas em linha de comando (CLI) projetada para otimizar fluxos operacionais. Desenvolvido na linguagem de programação C, o software visa auxiliar usuários na organização de suas atividades através da divisão estruturada de tarefas em três estados lógicos sequenciais: A Fazer, Em Andamento e Concluído.

O programa utiliza estruturas de dados estáticas para armazenar textos, oferecendo uma interface interativa baseada em um menu numérico que opera em ciclo contínuo até que a ação de encerramento seja explicitamente solicitada pelo usuário.

2. Explicação das Funcionalidades Implementadas

O sistema é modularizado internamente por meio de funções auxiliares de validação e blocos lógicos estruturados dentro de um menu principal (switch-case). Abaixo estão descritas as funcionalidades:

Funções Auxiliares de Validação

- **vaziaespaco(char str[]):** Analisa a string inserida pelo usuário para evitar o cadastro de tarefas sem identificação. Retorna verdadeiro (1) se a string for vazia ou composta exclusivamente por espaços em branco e tabulações (\t).
- **repetido(char matriz[][51], int qtd, char str[]):** Garante a unicidade das tarefas dentro do escopo do estado "A Fazer", comparando a nova entrada com os registros existentes por meio da função strcmp.

Funcionalidades do Menu Principal

- **Cadastrar Atividade:** Permite o input em lote de novas tarefas. O sistema solicita a quantidade desejada, valida se há espaço disponível na matriz estática (limite de 20) e

captura os nomes usando fgets. O caractere de nova linha (\n) é limpo via strcspn. Entradas inválidas (vazias ou duplicadas) são rejeitadas em um laço do-while.

- **Listar Atividades:** Exibe na tela, de forma categorizada e numerada, todas as atividades presentes em cada uma das três etapas do fluxo.
- **Movimentar Atividades:** Realiza a transição de estado das tarefas. O usuário possui duas rotas unidirecionais: A Fazer para Em Andamento ou Em Andamento para Concluído. Ao escolher a tarefa pelo seu índice, o sistema a copia para a matriz do próximo estágio e reorganiza a matriz de origem, rotacionando os elementos subsequentes uma posição para trás para não deixar lacunas ("gaps").
- **Consultar Quantidade:** Apresenta uma visão quantitativa do sistema, exibindo o total de tarefas alocadas em cada lista individualmente e o somatório geral do ecossistema.
- **Buscar Atividade:** Implementa uma busca por subsequência de caracteres (substring). Utilizando a função strstr, o sistema varre as três matrizes e informa ao usuário se a atividade foi encontrada, discriminando em qual etapa ela se encontra.
- **Sair:** Interrompe a execução do laço principal e encerra o programa com segurança.

3. Processo de Desenvolvimento e Linha de Raciocínio

A equipe estruturou a solução focando na simplicidade e na prevenção de erros comuns de manipulação de strings em C. O raciocínio lógico seguiu os seguintes pilares:

1. **Modelagem de Dados Estática:** Optou-se por utilizar matrizes bidimensionais de caracteres (char[linhas][colunas]) para representar as listas. O limite de caracteres foi fixado em 50 úteis (mais o terminador \0). Os tamanhos máximos foram definidos como 20 para as etapas iniciais e 40 para a etapa final, prevendo o acúmulo natural de itens concluídos.
2. **Gerenciamento de Estados:** Em vez de mover ponteiros ou criar estruturas complexas de nós, o grupo utilizou contadores inteiros isolados (qtd_afazer, qtd_andamento, qtd_concluido) para controlar o índice de inserção e o tamanho atualizado de cada lista.
3. **Lógica de Exclusão por Deslocamento (Shift):** Para remover um elemento de uma lista mantendo a ordem dos demais, a equipe implementou um algoritmo de rearranjo: o elemento selecionado é sobrescrito pelo seu sucessor direto, e esse comportamento replica-se até o final da lista ativa para não deixar lacunas.

4. Avaliação Crítica do Projeto e Melhorias Futuras

Pontos Fortes

- **Robustez na Entrada de Dados:** O tratamento sanitário de strings impede que entradas nulas, vazias ou poluídas com espaços em branco gerem falhas de renderização ou quebras na interface de linha de comando.
- **Busca Parcial Flexível:** A integração da função `strstr` confere alta usabilidade ao software, permitindo que o usuário localize registros por correspondência parcial sem a rigidez de correspondência exata.
- **Interface Intuitiva e Menu Contínuo:** A arquitetura CLI baseada em loops iterativos e seletores estruturados confere previsibilidade e facilidade de navigation para usuários de qualquer nível técnico.
- **Eficiência em Consumo de Memória Stack:** Ao utilizar limites estáticos bem dimensionados para o escopo proposto, o sistema elimina riscos de vazamento de memória (memory leaks) que ocorrem frequentemente em alocações dinâmicas mal gerenciadas.
- **Modularização Funcional de Regras Críticas:** A separação das validações complexas em rotinas utilitárias dedicadas (como `vaziaespaco` e `repetido`) demonstra boas práticas de coesão de código e reaproveitamento lógico.

Pontos Fracos e Vulnerabilidades Identificadas

- **Volatilidade dos Dados:** A ausência de escrita em memória não-volátil implica na perda total dos dados manipulados assim que a sessão do programa é encerrada pelo terminal.
- **Rigidez na Alocação Estática:** Os limites fixos impossibilitam a escalabilidade vertical do ecossistema, limitando o volume máximo de fluxo operacional a 20 tarefas simultâneas nos estágios primários.

Bug Crítico na Validação de Movimentação

Aviso: Nas linhas de verificação das opções de mover (case 1 e case 2 do menu de movimentação), a condition aceita a entrada `move >= 0`. Caso o usuário digite 0, o código tentará acessar o índice `move - 1`, resultando em um índice negativo inválido (-1), o que gera comportamento indefinido (undefined behavior) por invasão de memória. A validação correta deveria ser estritamente maior que zero (`move > 0`).

Sugestões para Melhorias Futuras

Mapeamento de soluções para superação dos pontos fracos identificados e otimização geral da arquitetura de software:

Área	Melhoria Proposta	Benefício
Persistência de Dados	Integração com manipulação de arquivos (fopen, fprintf) para salvar/carregar o estado das tarefas em arquivos .txt.	Permite o uso contínuo do sistema entre diferentes sessões de execução.
Estrutura de Dados	Substituição de matrizes estáticas por Listas Encadeadas Dinâmicas (malloc/free).	Elimina o limite rígido de tarefas, adaptando o consumo de memória sob demanda.
Correção de Bugs	Ajustar a verificação de limites para <code>if(move > 0 && move <= qtd_afazer)</code> .	Elimina o risco de falha de segmentação (Segmentation Fault).
Arquitetura de Código	Refatoração de cada case do menu em funções isoladas (ex: <code>void movimentarTarefa()</code>).	Melhora a manutenibilidade, legibilidade e organização do código-fonte.